

PROCESAMIENTO DE DATOS MASIVOS

Actividad 1 – Programación en Hadoop

Ejercicio 1

Enunciado

Dado un dataset que contenga entradas con la forma “persona;método_pago;dinero_gastado” **crea un programa llamado personaGastosSinTarjetaCredito que para cada persona indique la suma del dinero gastado con cualquier forma de pago exceptuando tarjeta de crédito**, con el formato persona;gastosinTDC.

Solución

Se propone la solución contenida en la carpeta “ej1”, compuesta por los siguientes documentos:

- Datos de entrada: casoDePrueba1.txt
- Código: se han desarrollado el script personaGastosSinTarjetaCredito.py
- Comando de ejecución: el código a escribir en la terminal se reporta en terminal_personaGastoSinTarjetaCredito.txt
- Resultado: el resultado generado por el programa se reporta en la carpeta personaGastosSinTarjetaCredito

Explicación Código Script

```
C: > Users > DANI > Downloads > AG_1-03MBID-Lopez-Daniel-Gorostiza-Ania > AG_1-03MBID-Lopez-Daniel-Gorostiza-Ania > ej1 >

1  from pyspark import SparkContext
2
3  def parse_line(line):
4      parts = line.strip().split(";")
5      if len(parts) == 3:
6          persona, metodo, gasto = parts
7          return (persona, metodo, float(gasto))
8      else:
9          return None
10
11 def not_none(record):
12     return record is not None
13
14 def gasto_sin_tarjeta(record):
15     persona, metodo, gasto = record
16     if metodo == "Tarjeta de crédito":
17         return (persona, 0.0)
18     else:
19         return (persona, gasto)
20
21 def sum_gastos(a, b):
22     return a + b
23
24 def format_output(record):
25     return f"{record[0]},{int(record[1])}"
26
27 if __name__ == "__main__":
28     sc = SparkContext(appName="personaGastosSinTarjetaCredito")
29
30     lines = sc.textFile("hdfs:///user/dani/datasets/casoDePrueba1.txt")
31
32     data = lines.map(parse_line).filter(not_none)
33     print("Particiones en data:", data.getNumPartitions())
34
35     gastos_sin_tdc = data.map(gasto_sin_tarjeta)
36     print("Particiones en gastos_sin_tdc:", gastos_sin_tdc.getNumPartitions())
37
38     suma_por_persona = gastos_sin_tdc.reduceByKey(sum_gastos)
39     print("Particiones en suma_por_persona:", suma_por_persona.getNumPartitions())
40
41     salida = suma_por_persona.map(format_output)
42
43     salida.saveAsTextFile("hdfs:///user/dani/output/gastos_sin_tarjeta4")
44
45     sc.stop()
```

El programa está diseñado en *PySpark* para procesar un archivo de texto almacenado en HDFS, donde cada línea representa un gasto realizado por una persona. El archivo tiene datos separados por punto y coma, en el formato “persona;metodo;gasto”. Por ejemplo, una línea podría ser “Alice;Bizum;200”. El objetivo es calcular cuánto ha gastado cada persona **sin** utilizar la tarjeta de crédito.

Todo comienza creando un contexto de *Spark*, necesario para ejecutar operaciones distribuidas sobre el cluster. El primer paso es leer el archivo con *textFile*, lo que crea el RDD *lines*. Este RDD contiene simplemente líneas de texto, cada una como un *string*, idéntico al contenido original del archivo.

A continuación, cada línea de texto es transformada en una estructura más útil: una tupla con tres elementos: persona, método de pago y gasto. Esto se logra aplicando *map* sobre *lines* usando la función de *parsing*. El resultado es el RDD *data*, cuyas entradas son tuplas como (“Alice”, “Bizum”, 200.0). Para garantizar que no existan registros inválidos, se filtran aquellos valores nulos que podrían haber resultado de líneas mal formadas en el archivo. Por tanto, *data* contiene exclusivamente tuplas correctas, con persona, método y gasto en número flotante.

Después, el programa necesita excluir los gastos hechos con tarjeta de crédito. Para ello, se transforma el RDD *data* en **gastos_sin_tdc**. Esto se consigue recorriendo cada registro y, si el método de pago es “Tarjeta de crédito”, se devuelve una tupla con la persona y un gasto de cero. Si el método de pago es diferente, se conserva el gasto original. Así, en **gastos_sin_tdc** se tiene un RDD con pares (persona, gasto), donde los gastos por tarjeta aparecen como cero, mientras que los demás gastos mantienen su valor real.

El siguiente paso es sumar todos esos gastos por persona. Esto se logra mediante la acción *reduceByKey*. Sobre el RDD **gastos_sin_tdc**, se agrupan los gastos que tengan la misma persona como clave, sumando sus importes. El resultado es el RDD **suma_por_persona**, cuyos registros son tuplas (persona, gasto_total), en las que el gasto total es la suma de todos los gastos de esa persona, excluyendo los de tarjeta de crédito gracias a que estos se sustituyeron por cero previamente. Así, por ejemplo, si Alice gastó 200 euros con Bizum y 250 euros con tarjeta, el resultado reflejará únicamente los 200 euros.

Para preparar los datos para guardarlos en HDFS, se transforma **suma_por_persona** en el RDD *salida*. Aquí, cada tupla se convierte en una línea de texto con formato “persona;gasto_total”, redondeando el gasto a número entero. Finalmente, se utiliza *saveAsTextFile* para escribir salida en HDFS, generando archivos de texto que almacenan los resultados, listos para ser consultados o utilizados en otros procesos. Por último, el contexto *Spark* se cierra para liberar los recursos del cluster.

Ejemplo:

```
[  
  "Alice;Tarjeta de crédito;250",  
  "Alice;Bizum;200",  
  "Bob;Efectivo;100",  
  "Charlie;Tarjeta de crédito;150"  
]
```

- Aplicamos `.map(parse_line)` y `.filter(not_none)`. RDD data

```
[  
  ("Alice", "Tarjeta de crédito", 250.0),  
  ("Alice", "Bizum", 200.0),  
  ("Bob", "Efectivo", 100.0),  
  ("Charlie", "Tarjeta de crédito", 150.0)  
]
```

- Aplicamos `.map(gasto_sin_tarjeta)`. RDD gastos_sin_tdc

```
[  
  ("Alice", 0.0),  
  ("Alice", 200.0),  
  ("Bob", 100.0),  
  ("Charlie", 0.0)  
]
```

- Aplicamos `.reduceByKey(sum_gastos)`, que suma los gastos por persona. RDD suma_por_persona

```
[  
  ("Alice", 200.0),  
  ("Bob", 100.0),  
  ("Charlie", 0.0)  
]
```

- **Aplicamos `.map(format_output)`. RDD salida.**

```
[  
  "Alice;200",  
  "Bob;100",  
  "Charlie;0"  
]
```

- **Finalmente, el RDD salida se escribe como archivos de texto en HDFS. El contenido final del archivo será:**

```
Alice;200  
Bob;100  
Charlie;0
```

Ejecución

Seguimos los siguientes pasos para ejecutar el programa:

1. Ubicarnos en la ruta deseada (cd /home/dani/)
2. Dar permiso de ejecución a los scripts (chmod u+x personaGastosSinTarjetaCredito.py)
3. Activamos el yarn: start-yarn.sh y el DFS: start-dfs.sh
4. Ejecutar el programa:

```
dani@dani-VMware-Virtual-Platform:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
dani@dani-VMware-Virtual-Platform:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [dani-VMware-Virtual-Platform]
```

```
dani@dani-VMware-Virtual-Platform:~$ spark-submit --master yarn personaGastosSinTarjetaCredito.py
25/07/09 18:52:50 WARN Utils: Your hostname, dani-VMware-Virtual-Platform resolves to a loopback address: 127.0.1.1; using 192.168.176.131 instead (on interface ens33)
25/07/09 18:52:50 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
25/07/09 18:52:51 INFO SparkContext: Running Spark version 3.4.4
25/07/09 18:52:51 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
25/07/09 18:52:51 INFO ResourceUtils: =====
=====
25/07/09 18:52:51 INFO ResourceUtils: No custom resources configured for spark.driver.
25/07/09 18:52:51 INFO ResourceUtils: =====
=====
25/07/09 18:52:51 INFO SparkContext: Submitted application: personaGastosSinTarjetaCredito
25/07/09 18:52:51 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map(cores -> name: cores, amount: 1, script: , vendor: , memory -> name: memory, amount: 1024, script: , vendor: , offHeap -> name: offHeap, amount: 0, script: , vendor: ), task resources: Map(cpus -> name: cpus, amount: 1.0)
25/07/09 18:52:51 INFO ResourceProfile: Limiting resource is cpus at 1 tasks per executor
```

Pasamos los datos obtenidos del Hadoop File System a local:

```
dani@dani-VMware-Virtual-Platform:~$ hdfs dfs -get /user/dani/output/personaGastosSinTarjetaCredito /home/dani/resultados
dani@dani-VMware-Virtual-Platform:~$
```

Resultado

Alice;200

Bob;0

Luis;300